

Sprint Documentation

Sprint #12	Start Date: 25/02/2026	Final Date: 09/03/2026
Projetc Name:	civitas-backend	
Ano: 5º	Análise e Desenvolvimento de Sistemas - AMS	
Team Members		
Wendell Alves Nascimento		
João Antônio Gomes Cestonaro		

Sprint Backlog

Task#	Description	Start Date	Final date
051	Estudo de Escalabilidade do Código	25/02/26	09/03/26

Task Description

Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
051	O objetivo desta sprint é realizar um estudo aprofundado de escalabilidade e desempenho do backend. Considerando que todos os CRUDs já estão implementados, esta etapa visa analisar gargalos, propor melhorias estruturais e preparar o sistema para lidar com: - Alto volume de requisições simultâneas - Grandes volumes de dados financeiros - Consultas complexas e relatórios extensos - Crescimento contínuo da base de dados O foco principal é performance, estabilidade e capacidade de crescimento sustentável.	João Gomes Cestonaro, Wendell Alves Nascimento.	Completo	18 horas	18 horas

Sprint Description

Tarefa: Sprint 12 – Estudo de Escalabilidade do Código

Objetivo

O objetivo desta sprint é realizar um estudo aprofundado de escalabilidade e desempenho do backend.

Considerando que todos os CRUDs já estão implementados, esta etapa visa analisar gargalos, propor melhorias estruturais e preparar o sistema para lidar com:

- Alto volume de requisições simultâneas
- Grandes volumes de dados financeiros
- Consultas complexas e relatórios extensos
- Crescimento contínuo da base de dados

O foco principal é performance, estabilidade e capacidade de crescimento sustentável.

Critérios de Aceite

- Análise da arquitetura atual do backend

Identificação de gargalos de desempenho (API, banco de dados, queries, rede)

Estudo de estratégias de escalabilidade vertical e horizontal

Proposta de melhorias estruturais e técnicas

Documentação das recomendações técnicas

Exemplos práticos aplicáveis ao contexto do sistema

Observações

- Priorizar desempenho em consultas de grande volume (relatórios financeiros, filtros por período, secretaria, fornecedor, etc.).
- Considerar ambiente de produção com múltiplos usuários simultâneos (servidores internos, transparência pública, órgãos fiscalizadores).
- Avaliar impacto tanto em infraestrutura quanto em código.

Sprint Results

Análise Consolidada de Escalabilidade e Desempenho

1. Visão Geral da Arquitetura Atual

O backend está estruturado em camadas bem definidas:

Controller/ Service/ Repository/ EF Core

A injeção de dependência centralizada no Program.cs demonstra uma base sólida para evolução e manutenção. A arquitetura é adequada para crescimento, porém existem pontos técnicos que podem se tornar gargalos conforme o volume de dados e usuários aumentar.

2. Principais Gargalos Identificados

2.1 Banco de Dados

2.1.1 Armazenamento de arquivos binários (Blob)

O modelo Documento armazena arquivos digitalizados em byte[] diretamente no banco.

Impactos:

- Alto consumo de memória na API
- Aumento do tempo de resposta
- Crescimento acelerado do banco
- Backup mais pesado
- Maior I/O de disco

2.1.2 Falta de Paginação

Existem endpoints GetAll que retornam listas completas.

Impacto direto:

- Latência cresce linearmente com o volume de registros
- Alto consumo de memória
- Possível timeout em grandes volumes

2.1.3 Índices Insuficientes

Filtros frequentes como:

- Ano
- Instituição
- Fornecedor
- Situação
- Operação
- Datas

podem não estar indexados adequadamente.

Consequência:

Consultas tornam-se lentas à medida que a base cresce.

2.1.4 N+1 Queries e Tracking Desnecessário

- Possível carregamento ineficiente de relacionamentos
- Consultas de leitura sendo executadas com tracking habilitado
- Ausência de AsNoTracking() em consultas que não alteram dados

Isso aumenta o consumo de memória e processamento desnecessariamente.

2.1.5 Tipagem Inadequada para Dados Financeiros e Datas

Foram identificados:

- Uso de double e float para valores monetários
- Datas armazenadas como string

Problemas causados:

- Perda de precisão financeira
- Dificuldade para ordenação e filtros
- Índices menos eficientes
- Relatórios inconsistentes

2.2 Camada de API

2.2.1 Ausência de Cache

Dados estáticos ou pouco mutáveis (ex: tipos, unidades, classificações) são consultados repetidamente.

Isso gera:

- Sobrecarga desnecessária no banco
- Aumento de latência

2.2.2 Serialização de Grandes Volumes

DTOs retornando dados completos sem projeções específicas por endpoint.

Impacto:

- Payload maior que o necessário
- Maior consumo de banda
- Tempo maior de serialização

2.2.3 Ausência de Compressão HTTP

Respostas não comprimidas aumentam:

- Tempo de transferência
- Uso de rede

- Latência percebida

2.2.4 Possíveis Ambiguidades de Rotas

Duplicidade de métodos ou rotas pode gerar:

- Conflitos futuros
- Dificuldade de manutenção
- Problemas de versionamento

3. Estratégias de Escalabilidade

3.1 Escalabilidade Vertical (Curto Prazo)

Recomendações mínimas para ambiente de produção:

- **CPU:** 4 cores dedicados
- **Memória RAM:** mínimo 8GB (ideal 16GB se houver cache intensivo)
- **Disco:** SSD para banco de dados (redução significativa de I/O)
- Configuração de pool de conexões otimizado
- Uso de AddDbContextPool

Essa abordagem é suficiente para crescimento inicial com baixo custo operacional.

3.2 Escalabilidade Horizontal (Médio e Longo Prazo)

Preparar a aplicação para:

- API Stateless
- Múltiplas réplicas (Load Balancer)
- Cache distribuído (ex: Redis)
- Read replicas no banco para relatórios pesados
- Separação entre carga transacional e analítica

Essa estrutura permite escalar conforme demanda real de usuários.

4. Melhorias Estruturais Prioritárias

Alta Prioridade

1. Implementar paginação obrigatória em todos os GetAll
2. Separar documento em:
 - Endpoint de metadados
 - Endpoint exclusivo para download binário
3. Migrar valores financeiros para decimal

4. Migrar datas para DateTime / timestamp
5. Criar índices direcionados para filtros mais usados
6. Implementar compressão HTTP (GZIP ou Brotli)

Média Prioridade

1. Aplicar AsNoTracking() em consultas de leitura
2. Implementar cache para dados estáticos
3. Criar projeções específicas de DTO por endpoint
4. Revisar rotas duplicadas
5. Ajustar pool de conexões

Evolução Estrutural

1. Armazenar arquivos digitalizados externamente (ex: storage dedicado) e manter apenas referência no banco
2. Implementar cache distribuído
3. Criar separação de leitura e escrita (read replicas)
4. Introduzir observabilidade completa:
 - Health checks
 - Métricas p95 / p99
 - Logs estruturados
 - Monitoramento de queries lentas

5. Métricas de Performance Recomendadas

Para manter o sistema saudável conforme cresce:

- Tempo médio de leitura: **< 200ms**
- Tempo médio de escrita: **< 500ms**
- Taxa de cache hit: **> 80%**
- Queries acima de 1 segundo devem ser analisadas
- Uso de memória abaixo de 70% da RAM disponível
- Pool de conexões ajustado conforme carga (ex: min 10 / max 100)

6. Resultado Esperado com as Implementações

Com as melhorias aplicadas, espera-se:

- Redução significativa da latência em consultas grandes

- Menor consumo de memória da API
- Maior throughput sob alta concorrência
- Banco de dados mais estável e previsível
- Base preparada para crescimento sustentável
- Melhor experiência para o usuário final

Conclusão

A arquitetura atual do sistema é bem estruturada e possui base sólida para evolução.

Entretanto, as melhorias propostas são fundamentais para evitar que o crescimento do volume de dados e usuários gere degradação progressiva de desempenho.

A implementação das ações priorizadas garante não apenas ganho imediato de performance, mas também prepara o sistema para escalar de forma segura, sustentável e profissional.

Assinaturas